(2) { ① 300 → Longest increasing Subsequence.
② 1143 → Longest Common Subsequence

③ 516 → Longest Palindromic Subsequence (LCS)

④ 1312 → (LPS, LCS)

(2) { ⑤ 5 → Longest Palindromic Substring.
⑥ 72 → edit distance.

(1) { ⑦ 44 → wildcard Matching

(2) { ⑧ 494 → Target Sum → Count $S_1 - S_2 = D$
⑨ 416 → Partition equal Subset Sum → Subset Sum equals to target

Subset Sum equal to target | Partition equal Subset Sum

Wildcard Matching :→

S_1 = "?ay"

S_2 = "ray"

? = = r

return true.

? → Matches with single character

* → Matches with sequence of length 0 or more.

S_1 = "ab*cd"

S_2 = abdefcd

'* = = def' return true

S_1 = * * abcd

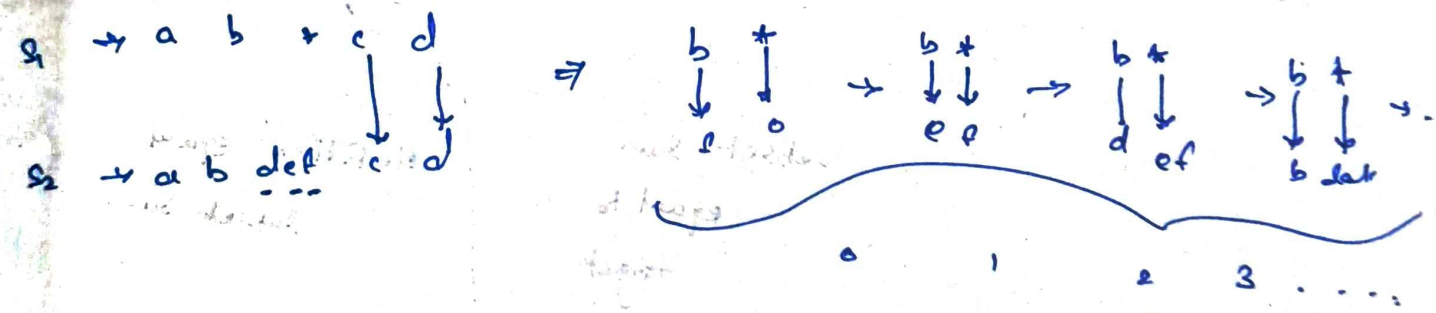
S_2 = abcd

0 0 zero return true

S_1 = "ab?d"

S_2 = abcc

return false



all possible ways

↓

Recursion (Strategy by)

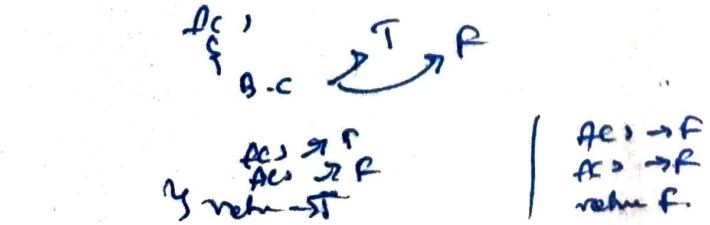
(*) Rules

(1) Empty (')')

(2) empty Comparison

(3) out of all Comparison, if anymore can, you return true.

→



(a) f(i,j) → f(n-1, m-1) → a b * c d

(b) f(4,6) a b d e f e d
0 1 2 3 4 5 6

→ $S_1[0 \ldots 4]$ and $S_2[0 \ldots 6]$ are they matching or not.

$S_1[0 \ldots m-1]$ and $S_2[0 \ldots m-1]$ are they matching or not

⇓ return

True / false

(c)

if the string matching then it only two possibility either they are matching or not.

if matching.

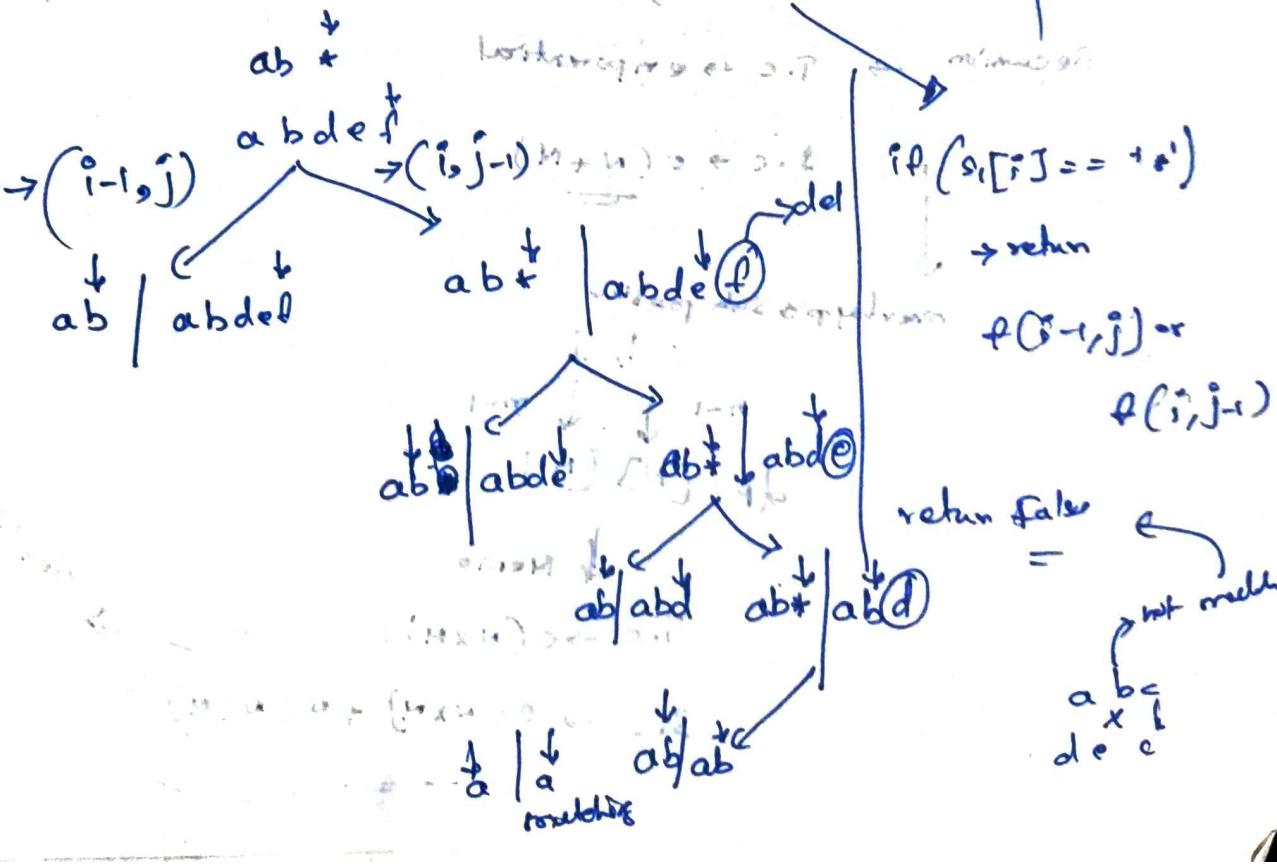
```
if ( S1[i] == S2[j] || S1[i] == '?' )
    return f(i-1, j-1);
```

while you are in start check for 0, 1, 2, 3, 4 ... length

i
0 1 2 3 4
a b * c d

j
a b c d e f e d
0 1 2 3 4 5 6

→



Base Case:→ return True or false

↓

if comparison has done

s_1 → has no more characters

s_2 → also no more characters

if s_1 gets is exhausted

$i \geq 0$ && $j \geq 0$ → for comparison to be successful. ①.

$i \geq 0$ && $j \geq 0$ → ②

if s_2 gets exhausted

$j \geq 0$ && $i \geq 0$

$s_1 =$ "something like a" ; → represent as empty

$s_2 =$ "a" ;

→ if s_1 is left, it has to be all '*'

```
for (i; i>=0 ; i--)
    if (s1[i] != '*')
        return false
}
```

Recursion → T.C → exponential

S.C → O(N+M)

↓

overlapping subproblem

i, j

n-1 ↓ ↓ m-1

dp [N] [M]

↓ Memo

T.C → O(N×M)

S.C → O(N×M) + O(N+M) → don't want

Tabulation: →

* Base Case
  - i =
  - j =
* Copy the Recurrence

i = 0, j = 0 } Convert the index acc. to 1 to n

Don't Convert the Memo Soln → 1 base index.

4/6

Qs on Subsequence / Subsets & target

[Contiguous / non-Contiguous] → 1, 3, 2 → 1, 2
3, 2

Subset Sum equal to K

arr → [1, 2, 3, 4] → k = 4

(1, 3)
4
true

→ Generate all subsequence & check if any of them gives Sum of k

Recursion
- (1) i, j → f(ind, target)
- (2) explore all
- (3) [redacted]

↓ return T/F.

a[ind] Part of the subsequence
OR
a[ind] not part of the Subsequence

(*) Base Condition

→ f(n-1, target)

↳ in the entire array fill the ind

target exist or not.

[1,2,3,4] += 4

f(3,4) ⇒ arr[0 ... 3] ⇒ does Someone exist in the target of 4

* → f(ind, target)

Base Condition

if (target == 0) return true;

if (ind == 0) return (a[0] == target)

return true;

(0 ⟵ n-1)

4

ind=0

arr[4]

bool not take = f(ind-1, target);

bool take = false

if (target >= a[ind])

take = f(ind-1, target-a[ind]);

0 ind } condition

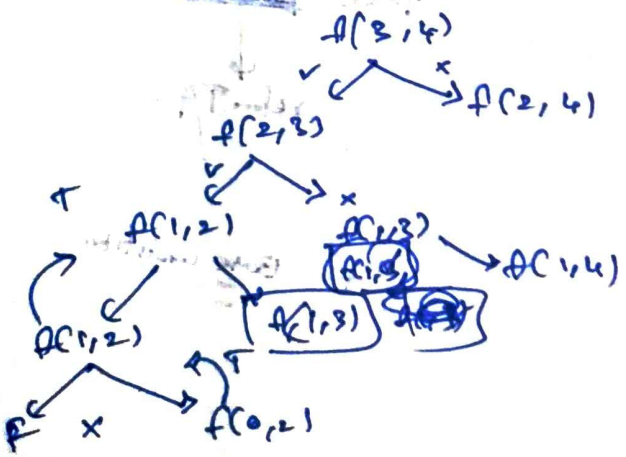
0 ind-1 ind X

condition

→ return (take) or (not take)

}

arr → [2, 3, 1, 1] target = 4



→ T.C O(2^n)

S.C → O(N)

↓

Overlapping Subproblems

Memoization

Tabulation :→

for (i=0 → n-1) → dp[i][0] = true;

dp[0][arr[0]] = true. → if (arr[0] == target)

→ ind → (1 → n-1)

→ target → (0 to target)

for ( )

{

for ( )

{

Copy

}

}

T.C → O(N × target)

S.C → O(N × target)

arr → of n integers

[2, 3, 3, 3, 4, 5]

[2, 3, 5] Subset 1

[3, 3, 4] Subset 2

divide the array into two subsets such sum $S_1 == S_2$

$S_1 = S_2$

$S_1 = S_2 = S/2$ → total sum

S

S_1 S_2

odd → ✗ not possible

even → (subset → sum S/2)

arr → [ ]

Subset Sum = S/2

S = total

LL →

arr[ ] , target

Subset

Return

odd → ans

S = total sum

false

LL

f(arr, S)

(Subset, S/2)

(#) Target Sum → dp + ↑ → dp + ↓ → dp + ↑
9.ms 88. mb 9. gb 16 mb

✓ (##) Coin change

✓ (3m) Longest Increasing Subsequence

✓ (#) Maximum product Subarray

(#) Perfect Squares

✓ (#) Minimum path Sum.

✓ (#) unique paths

✓ (#) unique paths 2

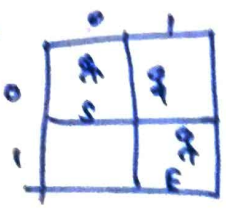
(#) dp on grids / 2D Matrix

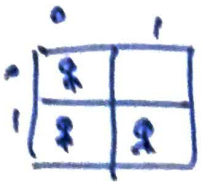
→ Count paths

→ Count paths with obstacles

→ Min path Sum.

unique path





either move

2×2 move.

(0,0) → (1,1)

2 unique ways

why Recursion? → (0,0) → (m-1, n-1)

finding all possible ways → so apply Recursion

How to write a Recursion?

① express anything in terms of index i, j. row col

② explore / do all paths

③ sum up all ways / max / min

(0,0) ⟶ (m-1, n-1)

f(i,j) → no. of unique ways (0,0) → (i,j)

↳ where should I start → f(m-1, n-1)

⊛ f(i,j)

{

if (i==0 && j==0)

return 1;

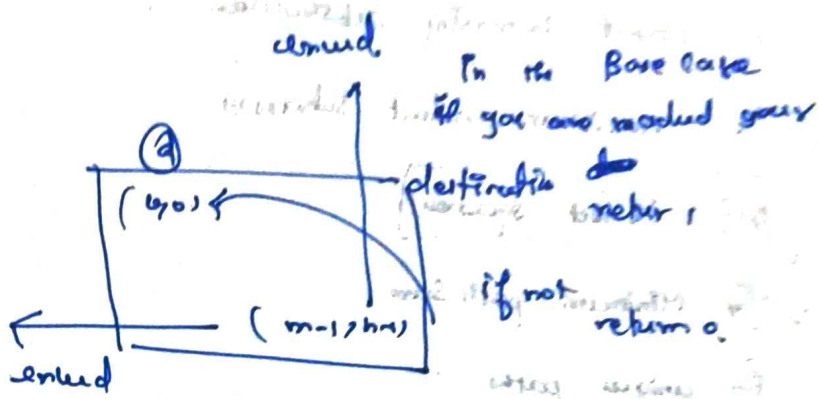
if (i<0 || j<0)

{

return 0;

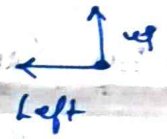
}

Base case.



explore { up = f(i-1, j)

left = f(i, j-1)



Sum { return up+left;

}

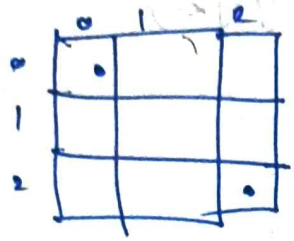
Recursion ⟶ T.C → $(2^{k+j}) \times n$ → exponential.

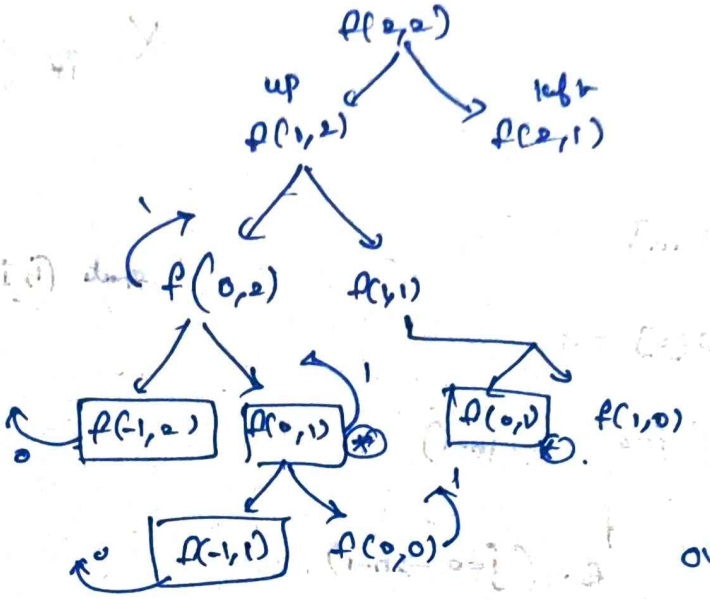
S.C → O(path length)

(m-1 + n-1)

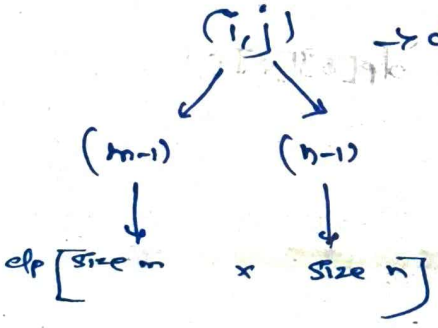
Recursion ⟶ DP ⟶ Memoization ⟶ If overlapping subproblems

m=3, n=3





Overlapping Subproblem



2D dp

How to Convert the Recursion to Memo :-

- declare the dp[m][n];
- store the ans
- check already solved if return dp[n]

→ T.C O(N)

O(N x M)

Linear

S.C → O(n-1 + m-1)

+

O(n x m)

Tabulation :→

```
f(i,j)
{
```

(*) dp[n][m]

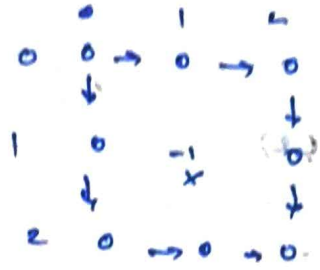
(*) dp[0][0] = 1

```
for ( i=0 → n-1)
{
  for ( j=0 → m-1)
  {
    if ( i==0 && j==0)   dp[0][0]=1
    else   up=0 , left=0
    {
      if(i>0)   ← up = dp[i-1][j]
      if(j>0)   ← left = dp[i][j-1]
      &
      dp[i][j]=up+left
    }
  }
}
```

✗ if (i<0 || j<0)

T.C → O(N×M)
S.C → O(N×M)

(6a) unique paths 2



(2) ways

```
f(i,j)
{
  if (i==0 && j==0)
      return 1;
  if (i<0 || j<0) return 0;
  up = f(i-1, j)
  left = f(i, j-1)
  return up + left
}
```

if dp[i][j] != -1 → dp[i][j] → Memo

dead cell

mat[i][j] == -1

can not go

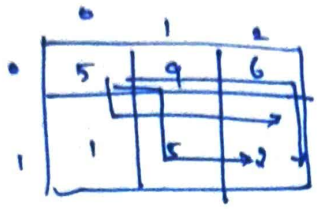
add of 1 more base case

```
if (i==0 && j==0 &&
    a[i][j]==-1)
  return 0;
```

Tabulation: →

```
for (i=0 → m-1)
{
for (j=0 → n-1)
{
  if (a[i][j] == -1) dp[i][j] = 0
  else if (i==0 && j==0) dp[i][j] = 1;
  else
    if (i>0) up = dp[i-1][j];
    if (j>0) left = dp[i][j-1];
    return left + left
}.
```

(62) Minimum path Sum :-



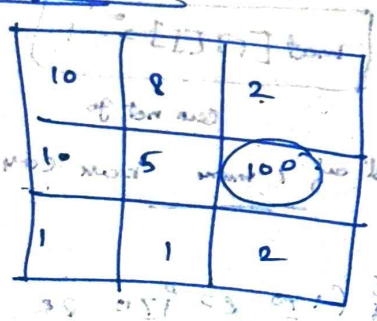
5 + 11 + 5 + 2

2 x 3

(0,0) → (1,2)

5+9+5+2 = (21) → min

why not follow greedy?



Try out all paths → R

Tell me the path which has min cost

(1) Empty (i,j)

(2) Explore all path

(3) Take the min paths.

f(i,j)

f(0,0) ⟶ f(m-1, n-1)

```
f(i,j)
{
  if (i==0 && j==0) return a[0][0];
  if (i<0 || j<0) return INT_MAX;
  up = a[i][j] + f(i-1, j);
  left = a[i][j] + f(i, j-1);
  return min(up, left);
}
```

Recursion → overlapping Subproblems → Memoization

(i, j)
↓ ↓
i-1 j-1
↓ ↓
dp[n × m] = -1

dp[i][j] = min (up, left)

if (dp[i][j] != -1) return dp[i][j];

T.C → O(N×M)

S.C → O(N×M) + O(path length)
↓
(m-1)+(n-1)

Tabulation: →

dp[n][m];

for (int i=0 → n-1)
{
for (j=0 → m-1)
{
if (i==0 && j==0) dp[i][j] = a[0][0];

else
{
if(i>0) up = a[i][j] + dp[i-1][j];
if (j>0) left = a[i][j] + dp[i][j-1];
dp[i][j] = min(up, left)
}
}
}
print (dp[n-1][m-1]);
}

→ array
(i,j)
0 → m-1
0 → m-1
→ copy the
same
→

(22) **Coin change:-**

arr[] → {1, 2, 3} → denomination of coins

target = 4

(3, 1) → 4 coins = 2 (How many coins)

(2, 2, 1) → 4 coins = 3 ✗

Greedy:- {1 2 3}

4/3 = 2 (3,

1/2 = 0 (3,

1/1 = 1 (3,1)

target = 4

It does wrong.

When does Greedy fail:-

{9, 6, 5, 1} → target = 11

11/9 = 1

2/1 = 2

{9, 1, 1}

3 Coins

{6, 5} → 2 Coins

Trying out all Combos to form target

Take the Combo that has min Coins

(R)

Rules:-

1. f(ind, T)
2. empty
3. Min of all